Backend Documentation – ExoHabitAI

1. Introduction

The backend system of the ExoHabitAI project is responsible for exposing the trained machine learning model through a REST API. The API allows users to send exoplanet parameters and receive habitability predictions.

The backend is built using the Flask framework. It loads a trained machine learning model and processes input data before generating predictions.

The backend performs the following tasks:

- Load trained machine learning model
- Accept user input through API
- Validate input parameters
- Perform prediction using ML model
- Return structured JSON response

2. Backend Architecture

The backend consists of the following components:

1. Flask Application

Handles HTTP requests and responses.

2. Machine Learning Model

The trained model is stored in .pkl format and loaded using joblib.

3. Input Validation Module

Ensures that incoming data contains all required parameters.

4. REST API Endpoints

Provides interfaces for prediction and ranking.

3. Backend Folder Structure:

```
backend/
 ├── app.py
 ├── utils.py

models/
 └── random_forest.pkl
```

File	Description
app.py	Main Flask server
utils.py	Input validation functions
models/random_forest.pkl	Trained ML model

4.2 Prediction Endpoint:

POST /predict

Description

Accepts exoplanet parameters and returns a habitability prediction.

Request Format

```
{
 "radius": 1.2,
 "mass": 1.1,
 "temperature": 288,
 "distance": 1.0
}
```

Parameter	Description
radius	Planet radius
mass	Planet mass
temperature	Planet surface temperature
distance	Distance from star

Response Format:

```
{
 "status": "success",
 "prediction": "Habitable",
 "probability": 0.82
}
```

Field	Description
status	API execution status
prediction	Habitability prediction
probability	Confidence score

4.3 Ranking Endpoint

Endpoint

POST /rank

Request Format

```
{
 "planets": [
  {
   "radius": 1.1,
   "mass": 1.0,
   "temperature": 290,
   "distance": 1.0
  },
  {
   "radius": 2.3,
   "mass": 1.9,
   "temperature": 350,
   "distance": 0.5
  }
 ]
}
```


Response Format


```
{
 "status": "success",
 "results": [
  {
   "planet_id": 1,
   "habitability_score": 0.91
  },
  {
   "planet_id": 2,
   "habitability_score": 0.52
  }
 ]
}
```

5. Input Validation

Input validation is implemented in utils.py.

The system checks for:

- Missing JSON input
- Missing parameters
- Invalid data types
- Incorrect values

If validation fails, the API returns an error response.

6. Machine Learning Model

The prediction system uses a trained **Random Forest classifier** implemented using the scikit-learn library.

The trained model is saved as:

random_forest.pkl

7. API Response Structure

All API responses follow a structured JSON format.

Success Response

```
{
 "status": "success",
 "data": {...}
}
```

Error Response

```
{
 "status": "error",
 "message": "Description of error"
}
```

8. Backend Testing

The backend APIs were tested using:

- Browser (for home route)
- Postman
- curl

Testing ensured that:

- APIs return correct responses
- Input validation works correctly
- Backend does not crash on invalid input

9. Conclusion

The backend successfully exposes the trained machine learning model through REST APIs. The system supports prediction and ranking of exoplanets based on habitability factors.

The backend is scalable and can be integrated with frontend applications or deployed to cloud platforms for real-world use.